Containerization and DevOps Lab

EXPERIMENT – 01

Name: ARYAN RAJ
Batch: B4
SAP ID: 500126035
Roll No: R2142231908

1 Objective

- To understand the conceptual and practical differences between Virtual Machines Containers.
- To install and configure a Virtual Machine using VirtualBox and Vagrant on Windows.
- To install and configure Containers using Docker inside WSL.
- To deploy an Ubuntu-based Nginx web server in both environments.
- To compare resource utilization, performance, and operational characteristics of VMs and Containers.

2 Theory

2.1 Virtual Machine (VM)

A Virtual Machine emulates a complete physical computer, including its own operating system kernel, hardware drivers, and user space. Each VM runs on top of a hypervisor.

Characteristics:

- Full OS per VM
- Higher resource usage
- Strong isolation
- Slower startup time

2.2 Container

Containers virtualize at the operating system level. They share the host OS kernel while isolating applications and dependencies in user space.

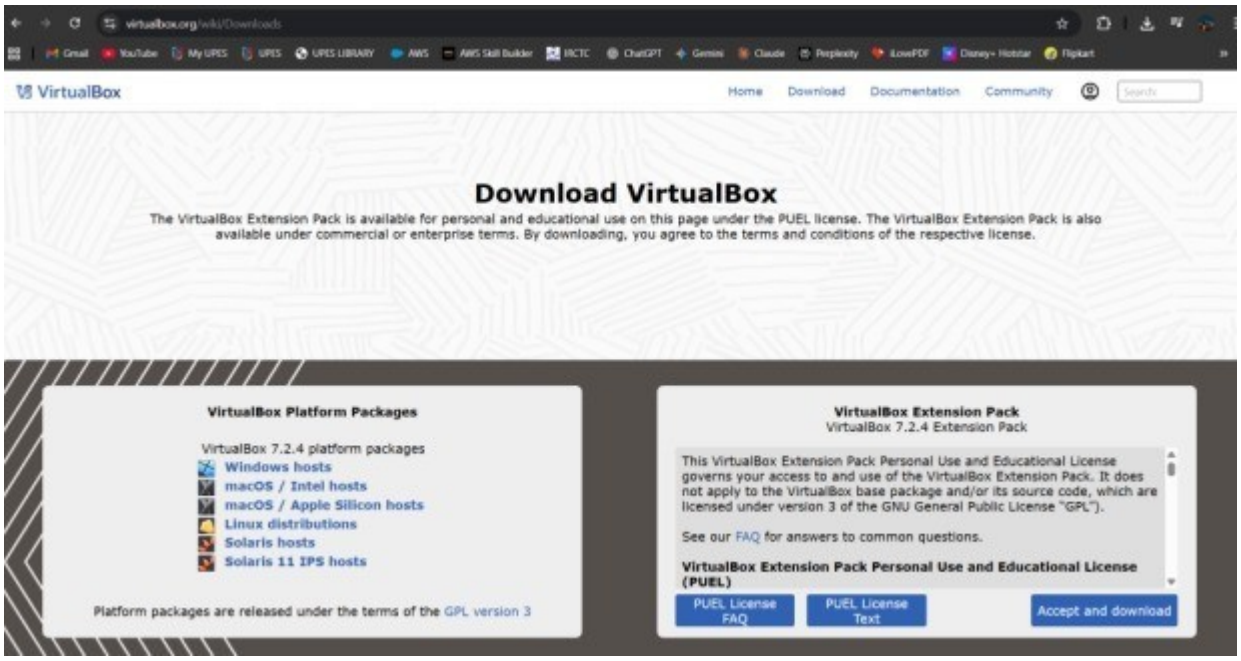
Characteristics:

- Shared kernel
- Lightweight
- Fast startup
- Efficient resource usage

3 Experiment Setup – Part A: Virtual Machine (Windows)

Step 1: Install VirtualBox

- Download VirtualBox from the official website.
- Run the installer with default settings.
- Restart the system if prompted.

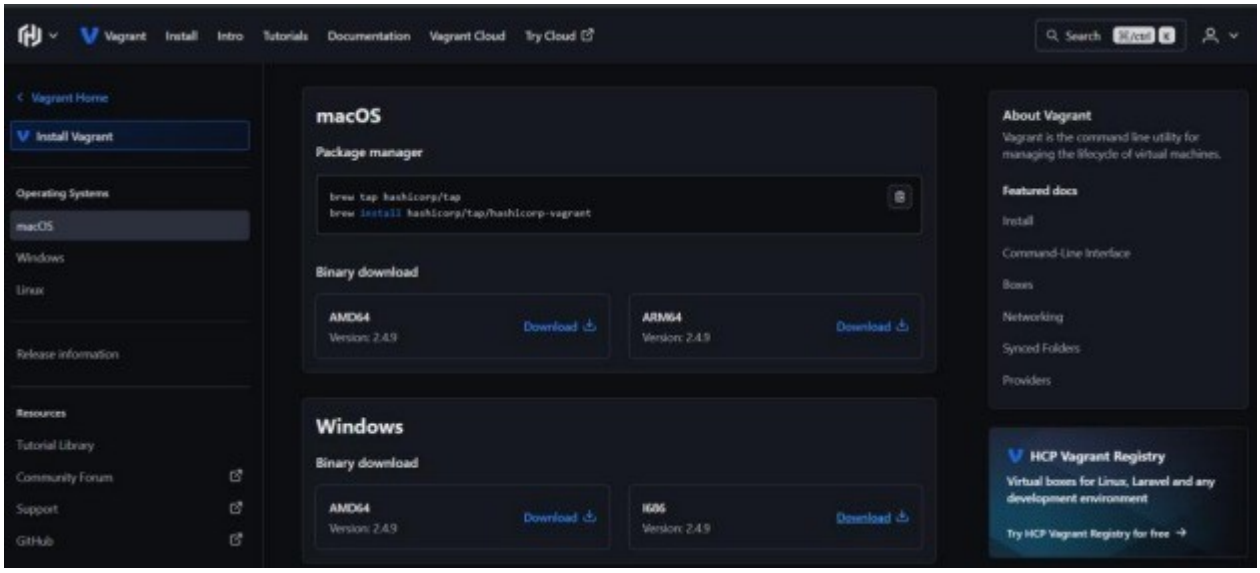


Step 2: Install Vagrant

- Download Vagrant for Windows.
- Install using default settings.

Verify installation:

```
vagrant --version
```



Step 3: Create Ubuntu VM using Vagrant

Initialize Vagrant:

```
vagrant init hashicorp/bionic64
```

Start the VM: `vagrant up`

Explanation:

vagrant init hashicorp/bionic64 creates a Vagrantfile in the project folder.
hashicorp/bionic64 refers to Ubuntu 18.04 LTS (64-bit).
vagrant up reads the Vagrantfile, downloads the image (if needed), allocates CPU/RAM, and boots the VM.

Access the VM:

```
vagrant ssh
```

This connects to the VM via SSH without requiring a password.

Step 4: Install Nginx inside VM

```
sudo apt update sudo apt install -y nginx sudo systemctl start nginx
```

Measure startup time:

```
time systemctl start nginx
```

Step 5: Verify Nginx

```
curl localhost
```

Stop and remove VM:

```
vagrant halt
vagrant destroy
```

4 Experiment Setup – Part B: Containers using WSL

Step 1: Install WSL 2

```
wsl --install
```

Verify installation:

```
wsl --version
```

```
PS C:\WINDOWS\system32> wsl --version
WSL version: 2.6.3.0
Kernel version: 6.6.87.2-1
WSLg version: 1.0.71
MSRDC version: 1.2.6353
Direct3D version: 1.611.1-81528511
DXCore version: 10.0.26100.1-240331-1435.ge-release
Windows version: 10.0.26200.7462
PS C:\WINDOWS\system32>
```

Step 2: Install Ubuntu on WSL

```
wsl --install -d
```

`Ubuntu` Verify: `wsl -l -v`

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\cloud sem 5\LAB sem6\lab 1> wsl -l -v
  NAME            STATE           VERSION
* Ubuntu-24.04    Stopped         2
PS D:\cloud sem 5\LAB sem6\lab 1> wsl
```

Step 3: Install Docker Engine inside WSL

```
sudo apt update sudo apt
install -y docker.io sudo
systemctl start docker sudo
usermod -aG docker $USER
```

Verify Docker:

```
docker --version
```

```
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket →
/usr/lib/systemd/system/docker.socket
Processing triggers for man-db (2.12.043) ...
Processing triggers for libc-bin (2.39-0ubuntu8.5) ...
Docker version 29.1.5, build 06efee6
AryanRaj@DESKTOP-B2CMN4K:/mnt/c/WINDOWS/ docker --vesion
Docker version 29.1.5, build 06efee6
AryanRaj@DESKTOP-B2CMN4K:/mnt/c/WINDOWS/ sudo apt install podman -y

AryanRaj@DESKTOP-B2CMN4K:/mnt/c/WINDOWS/system32$
Podman --version
Docker version 29.1.5, build 06efee6

AryanRaj@DESKTOP-B2CMN4K:/mnt/c/WINDOWS/system32$
Reading package lists... Done
Building dependency tree.. Done
Reading state information.. Done
The following additional packages will be installed:
  aardvark-dns buildah catatonit common containernetworking-plugins fuse-overlayfs
  golang-github-containers-common golang-github-containers-image libgpgme11t64 libsudi4
  netav
  passt uidmap

AryanRaj@DESKTOP-B2CMN4K:/mnt/c/WINDOWS/system32$
```

Step 4: Run Nginx Container

Pull image:

`docker pull ubuntu` Run container: `docker run -d -p`

`8080:80 --name nginx-container nginx`

```
tainer nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
500799c30424: Pull complete
10b68cfefee1: Pull complete
57f0dd1befe2: Pull complete
eaf8753feae0: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
390da5d749864f89cd67e67633131a2e0350b5f4f473878817ecb627fc0a983c
```

Explanation:

`docker pull ubuntu` downloads the Ubuntu image from Docker Hub.

`docker run -d -p8080:80--name nginx-containernginx` runs Nginx in detached mode and maps port 8080 to container port 80.

Step 5: Verify Nginx in Container

```
curl localhost:8080
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
```

5 Resource Utilization Observation

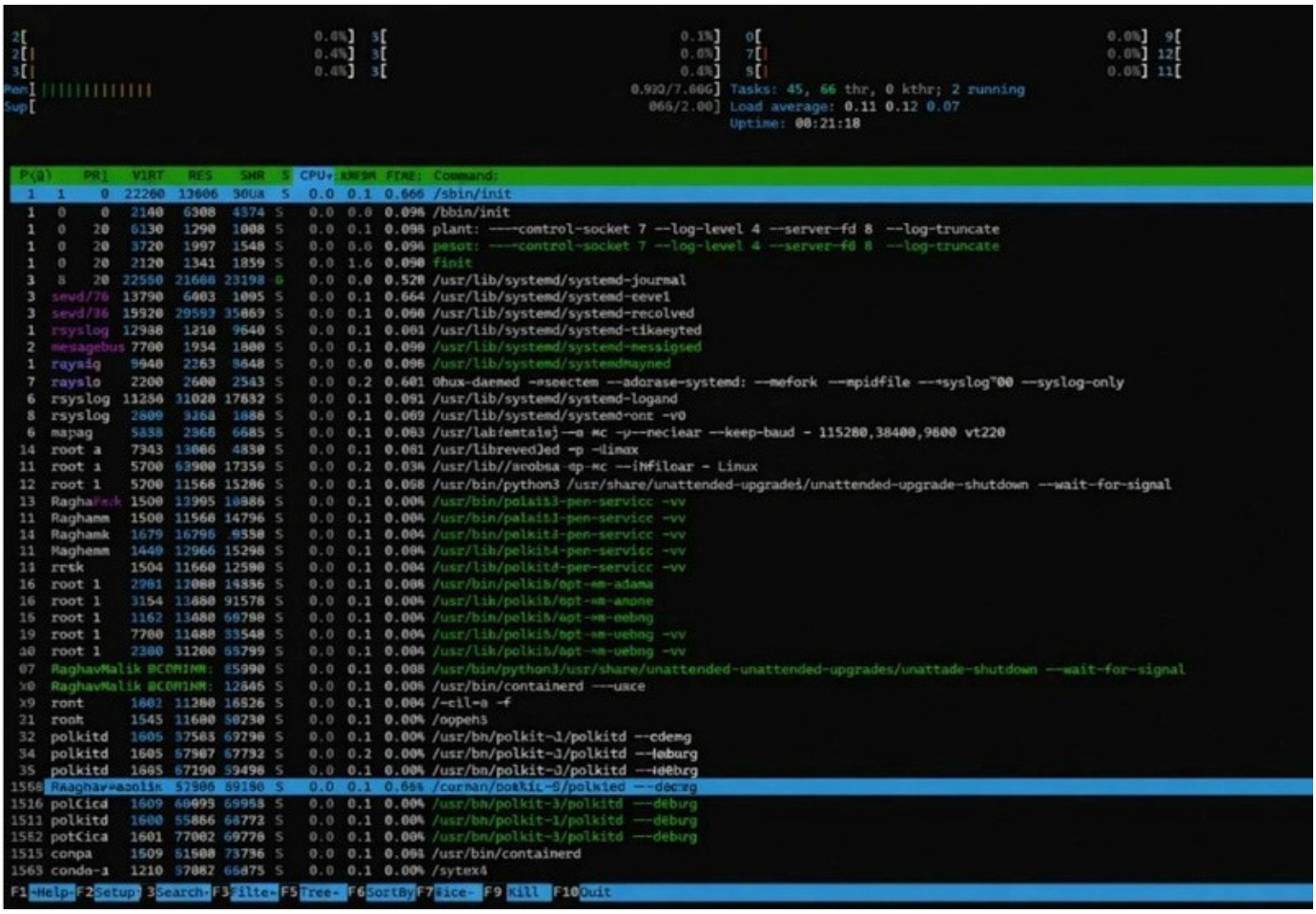
5.1 VM Observation Commands

Memory usage:

```
free -h
```

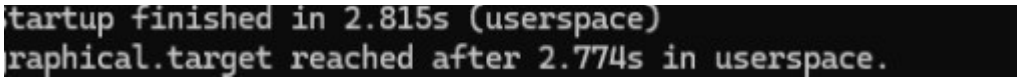
Real-time monitoring:

```
htop
```



Boot time:

```
systemd-analyze
```



5.2 Container Observation Commands

Container resource usage:

```
docker stats
```

```
CONTAINER ID   NAME              CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O
    PIDS
390da5d74986   nginx-container   0.00%     10.32MiB / 7.611GiB   0.13%     2.09kB / 1.4kB   0B / 12.3k
B   13
```

Memory usage:

```
free -h
```

```
[2]+  Stopped                 docker stats
aryanraj: ····—docker stats
aryanraj@B2CMN4K:/mnt/d/cloud sem 5/LAB
$$ free -h
               total        used        free      shared  buff/cache
Mem:           7.6Gi       606Mi       6.7Gi       3.7Mi       494Mi
Swap:          2.0Gi          0B       2.0Gi                   7.0Gi
aryanraj@B2CMN4K:/mnt/d/cloud sem 5/LAB sem1
```

6 Comparison: Virtual Machine vs Container

Parameter	Virtual Machine	Container
Boot Time	Slower (Full OS boot)	Very Fast
RAM Usage	High	Low
CPU Overhead	Higher	Minimal
Disk Usage	Large	Lightweight
Isolation	Strong	Moderate

7 Conclusion

Virtual Machines provide strong isolation and are suitable for legacy systems and full OS environments.

Containers are lightweight, start quickly, consume fewer resources, and are ideal for modern DevOps workflows and microservices architecture.